Machine Learning Assignment 2: Neural Networks

Francesco Gnan

Question 1

A two-layer neural network is defined by the following equation:

$$y_k := \mathbf{w}^{(2)} f\big(\mathbf{x}_k W^{(1)} + \mathbf{b}^{(1)}\big)^T + b^{(2)}$$

The values of the input vectors $\mathbf{x}_k$ (summarized in the matrix X), targets t_k (summarized in the vector $\mathbf{t}$), weights $W^{(1)}$ and $\mathbf{w}^{(2)}$ as well as biases $\mathbf{b}^{(1)}$ and $b^{(2)}$ are given below.

$$X = \begin{bmatrix} — & \mathbf{x}_1 & — \\ — & \mathbf{x}_2 & — \\ — & \mathbf{x}_2 & — \\ — & \mathbf{x}_2 & — \end{bmatrix} = [[0.6, -1.0], [0.8, -1.0], [-0.4, 0.9], [0.2, 0.0]]$$

$$\mathbf{t} = \begin{bmatrix} t_1 \\ t_2 \\ t_3 \\ t_4 \end{bmatrix} = [-0.8, -0.1, 0.9, 0.7]$$

$$W^{(1)} = [[-0.8, -0.7, 0.6], [-1.0, 0.5, -1.0]]$$

$$\mathbf{b}^{(1)} = [-0.2, -1.0, -0.7]$$

$$\mathbf{w}^{(2)} = [0.1, -1.0, 0.5]$$

$$b^{(2)} = -0.7$$

The function f is the ReLU nonlinearity. The loss of the model is

$$L := \frac{1}{2} \sum_{k=1}^{4} (y_k - t_k)^2.$$

We want to numerically calculate the gradients of L with respect to $W^{(1)}$, $\mathbf{w}^{(2)}$, $\mathbf{b}^{(1)}$ and $b^{(2)}$. Analytically these gradients have the following form:

$$\frac{\partial L}{\partial \mathbf{W}^{(1)}} = \frac{\partial L}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{f}} \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{W}^{(1)}}$$

$$\frac{\partial L}{\partial \mathbf{w}^{(2)}} = \frac{\partial L}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{w}^{(2)}}$$

$$\frac{\partial L}{\partial \mathbf{b}^{(1)}} = \frac{\partial L}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{f}} \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{b}^{(1)}}$$

$$\frac{\partial L}{\partial b^{(2)}} = \frac{\partial L}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial b^{(2)}}$$

where:

$$\mathbf{u} = \mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}$$

$$\frac{\partial L}{\partial \mathbf{y}} = \mathbf{y} - \mathbf{t}$$

$$\frac{\partial \mathbf{y}}{\partial \mathbf{f}} = \mathbf{w}^{(2)}$$

$$\frac{\partial \mathbf{f}}{\partial \mathbf{u}} = 0 \quad \text{if} \quad \mathbf{u} < 0, \quad 1 \quad \text{if} \quad \mathbf{u} > 0$$

$$\frac{\partial \mathbf{y}}{\partial \mathbf{w}^{(2)}} = \mathbf{f}(\mathbf{u})$$

$$\frac{\partial \mathbf{u}}{\partial \mathbf{W}^{(1)}} = \mathbf{X}$$

$$\frac{\partial \mathbf{u}}{\partial \mathbf{b}^{(1)}} = [1, 1, 1]$$

$$\frac{\partial \mathbf{y}}{\partial b^{(2)}} = 1$$

Thanks to the implementation in file `Q1.py` we got the following results:

$$L = 2.38554$$

$$\frac{\partial L}{\partial \mathbf{W}^{(1)}} = \begin{bmatrix} 0.0122 & 0 & 0.061 \\ -0.0268 & 0 & -0.134 \end{bmatrix}$$

$$\frac{\partial L}{\partial \mathbf{w}^{(2)}} = \begin{bmatrix} 0.1168 & 0 & 0.1536 \end{bmatrix}$$

$$\frac{\partial L}{\partial \mathbf{b}^{(1)}} = \begin{bmatrix} 0.0268 & 0 & 0.134 \end{bmatrix}$$

$$\frac{\partial L}{\partial b^{(2)}} = -2.7319999999999998$$

Question 2

Often the Softmax nonlinearity at the output neurons and the Crossentropy loss are combined into a single Softmax+Crossentropy loss function. With this loss function, the error is $E = -\sum_k t_k \log\left(\frac{\exp(y_k)}{\sum_i \exp(y_i)}\right)$, with y_k being the inputs to the softmax function (i.e. the outputs of the neural network), and t_k being the elements of the target vector. We want to derive and simplify the gradient $\frac{\partial E}{\partial y_i}$.

$$E = -\sum_k t_k \log\left(\frac{\exp(y_k)}{\sum_i \exp(y_i)}\right) = -\sum_k t_k \log g_k$$

where we defined $g_k = \frac{\exp(y_k)}{\sum_i \exp(y_i)}$.
Applying the chain rule:

$$\frac{\partial E}{\partial y_i} = -\sum_k t_k \frac{1}{g_k} \frac{\partial g_k}{\partial y_i}$$

Now we distinguish two cases:

1. if $i = k$:

$$\frac{\partial g_k}{\partial y_i} = \frac{\exp(y_i)\sum_i \exp(y_i) - \exp(y_i)\exp(y_i)}{(\sum_i \exp(y_i))^2} = \frac{\exp(y_i)}{\sum_i \exp(y_i)}\left(\frac{\sum_i \exp(y_i) - \exp(y_i)}{\sum_i \exp(y_i)}\right)$$

   Thus, $\frac{\partial g_k}{\partial y_i} = g_k(1 - g_k)$

2. if $i \neq k$:

$$\frac{\partial g_k}{\partial y_i} = -\frac{\exp(y_k)\exp(y_i)}{(\sum_i \exp(y_i))^2} = -\frac{\exp(y_k)}{\sum_i \exp(y_i)}\frac{\exp(y_i)}{\sum_i \exp(y_i)}$$

   Thus, $\frac{\partial g_k}{\partial y_i} = -g_k g_i$

So, putting the results together we get:

$$\frac{\partial E}{\partial y_i} = -\sum_k t_k \frac{1}{g_k} \begin{cases} g_k(1 - g_k), & \text{if } i = k \\ -g_k g_i & \text{if } i \neq k \end{cases} = \sum_k \begin{cases} t_k(g_k - 1), & \text{if } i = k \\ t_k g_i & \text{if } i \neq k \end{cases}$$

If the target is a one-hot vector $\mathbf{t} = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \end{bmatrix}$:

$$\frac{\partial E}{\partial y_i} = g_i - t_i$$

Question 3

If we choose a learning rate that is too high, gradient descent can diverge (even in the case of convex functions). We want to use gradient descent to find the value of x that minimizes $f(x) = 5x^2 + 2$. The update via gradient descent can be written as $x_{n+1} = x_n - \eta f'(x_n)$, with η being the learning rate. We want to find the range of values that η can take so that gradient descent converges to the minimum from any starting point.
In order to guarantee convergence from any starting point x_n, want that the next step $x_{n+1} = x_n - \eta f'(x_n) = x_n - \eta(10x_n)$ is such that:

- $x_{n+1} < x_n$
- $x_{n+1} > -x_n$

We want to find an interval for η that satisfies both these two conditions. Since $f(x)$ is a parable, the previous conditions guarantee that at each step we get closer and closer to the minimum of the parable itself.

- $x_{n+1} = x_n - \eta(10x_n) < x_n \rightarrow \eta > 0$
- $x_{n+1} = x_n - \eta(10x_n) > -x_n \rightarrow \eta < \frac{1}{5}$

So we found that the desired interval is $\eta \in (0, \frac{1}{5})$.